

From zero to interview-ready — every concept explained simply with real examples

1	What is Machine Learning? AI vs ML vs DL — the big picture
2	Types of Machine Learning Supervised · Unsupervised · Reinforcement
3	Overfitting & Underfitting The most common interview trap
4	Data Preprocessing Missing values · Scaling · Encoding · Feature Engineering
5	Supervised Algorithms Linear Reg · Logistic Reg · Decision Tree · RF · KNN · SVM · Naive Bayes
6	Unsupervised Algorithms K-Means · PCA · Hierarchical · Anomaly Detection
7	Evaluation Metrics Confusion Matrix · Accuracy · Precision · Recall · F1 · AUC · RMSE
8	Bias, Variance & Regularization Why models fail and how to fix them
9	Gradient Descent & Optimization How models actually learn
10	Neural Networks & Deep Learning ANNs · Activation Functions · CNN · RNN · LSTM
11	XGBoost — Deep Dive The algorithm powering VigilAI — inside out
12	NLP — Natural Language Processing Tokenization · TF-IDF · Sentiment · Text Classification
13	LLMs, RAG & Prompt Engineering GPT · Claude · Hallucination · Fine-tuning · Agents
14	Practical ML Imbalanced data · Data leakage · Hyperparameter tuning
★	Quick Revision + Top 30 Q&A; Everything distilled — read this before your interview

1

What is Machine Learning?

The big picture — AI, ML, DL and why they're different

DEFINITION**Machine Learning**

A way of programming computers to learn patterns from data automatically, instead of a human writing every rule manually. You give it examples — it figures out the rules itself.

The Simple Analogy

Think of it this way: Teaching a child to recognise dogs. You don't write a rulebook ('if 4 legs AND fur AND barks → dog'). Instead you show 10,000 photos labelled dog or not-dog and the child learns the pattern. ML works exactly the same way.

Traditional Programming vs Machine Learning

The fundamental difference in HOW the computer gets its answer:

- Traditional Programming: You write rules → computer follows them → output
- Machine Learning: You give data + correct answers → computer figures out the rules → model

Example — Spam Detection:

- Old way: manually write IF 'free money' in email THEN spam — breaks when spammers change wording
- ML way: show 100,000 labelled emails, model learns the pattern — adapts automatically to new spam

AI vs ML vs Deep Learning — Nested Circles

People use these three terms interchangeably — they are actually nested circles:

- AI (Artificial Intelligence) — the outermost, broadest circle
 - Any technique that enables a machine to simulate intelligent behaviour
 - Includes rule-based systems, search algorithms, robotics, AND machine learning
- ML (Machine Learning) — a subset inside AI
 - AI that specifically learns from data without being explicitly programmed with rules
- DL (Deep Learning) — a subset inside ML
 - ML using multi-layered Artificial Neural Networks (loosely inspired by the brain)
 - Needs massive data and GPU compute, but learns very complex patterns (images, speech, text)

Data Science is different — it overlaps with ML but also includes statistics, data analysis, visualisation, and domain expertise to extract business insights.

Term	One-Line Definition	Real Example
AI	Machines simulating intelligent behaviour	Chess engine, Siri, self-driving
ML	Computers learning from data automatically	Netflix recommendation, spam filter
Deep Learning	ML using many-layered neural networks	Face recognition, ChatGPT, image search
Data Science	Statistics + ML + domain knowledge to find insights	Business dashboards, A/B tests

■ Remember

- $DL \subset ML \subset AI$ — nested, not the same thing
- ML = computer learns rules from data. Traditional code = human writes the rules
- More data + right algorithm = better ML model

2

Types of Machine Learning

Supervised · Unsupervised · Reinforcement — with plain-English examples

DEFINITION**Supervised Learning**

Training a model on LABELLED data — every input comes with the correct answer. The model learns to predict the answer for new, unseen inputs.

Keyword: Labelled data. The 'supervisor' is the correct answer guiding learning.

- Input (features) + Correct Output (label) → model learns the mapping
- At prediction time: give new input → model predicts output

Real examples:

- Email → Spam or Not Spam (Classification)
- House size + location → Price in rupees (Regression)
- Patient symptoms → Disease risk score — exactly what you built in VigilAI!
- Photo → Cat or Dog

Think of it this way: Like a student studying with an answer key. Every practice question comes with the correct answer. The student (model) learns by comparing their guess to the answer key and correcting mistakes.

DEFINITION**Unsupervised Learning**

Training a model on data with NO labels — no correct answers provided. The model must find hidden patterns, groups, or structure in the data on its own.

Keyword: Unlabelled data. The model discovers structure nobody told it about.

Real examples:

- Group 10 million customers by buying behaviour — nobody told you how many groups exist
- Detect unusual bank transactions — nobody manually labelled all fraud cases
- Compress 100 features into 5 important ones (PCA)
- Find topics in 1 million news articles automatically

Think of it this way: Like a student given a stack of unlabelled photos and asked to sort them into groups. Nobody says 'these are cats, these are dogs' — the student figures out categories by noticing similarities.

DEFINITION**Reinforcement Learning**

An agent learns by taking actions in an environment and receiving rewards or penalties. It learns to maximise total reward over time through trial and error.

Key components:

- Agent — the AI that takes actions (e.g., a game-playing bot, a robot)
- Environment — the world the agent operates in (e.g., chess board, road)
- Action — what the agent does (e.g., move a piece, turn the steering wheel)
- Reward — positive feedback for good actions, negative for bad
- Policy — the strategy the agent learns: given this state → take this action

Real examples:

- AlphaGo — played millions of games against itself, learned winning chess/go moves
- Self-driving cars — rewarded for staying in lane, penalised for hitting objects
- Robot learning to walk — falls down → negative reward → adjusts → eventually walks

Think of it this way: Like training a dog. Give it a treat when it sits on command (positive reward), say 'no' when it misbehaves (negative reward). The dog (agent) learns which behaviours produce treats.

Type	Input Data	Goal	Real Example
Supervised	Data WITH labels (correct answers)	Predict output for new inputs	Disease detection, spam filter, VigilAI
Unsupervised	Data WITHOUT labels	Find hidden structure or groups	Customer segmentation, anomaly detection
Reinforcement	Rewards & penalties from actions	Maximise total reward over time	Game AI, robotics, self-driving cars

■ Remember

- Supervised = you provide labels (correct answers)
- Unsupervised = no labels, model finds patterns itself
- VigilAI uses Supervised Learning — symptoms (features) + risk score (label)
- Reinforcement Learning is rare in fresher interviews — know the concept

3

Overfitting & Underfitting

The most common interview topic — understand this deeply

The Core Goal: Generalisation

The entire goal of ML is to build a model that performs well on NEW, unseen data — not just the data it was trained on. This ability is called generalisation.

Two things can go wrong:

- Model learns TOO MUCH from training data → Overfitting (memorised, can't generalise)
- Model learns TOO LITTLE from training data → Underfitting (too simple, learns nothing)

DEFINITION**Overfitting**

The model learns the training data so well — including its noise and random quirks — that it fails to generalise to new data. Excellent on training, very poor on test data.

How to spot it:

- Training accuracy: 98% Test accuracy: 62% ← large gap = overfitting

Why it happens:

- Model is too complex (too many layers, too many features, too deep a tree)
- Too little training data — model memorises examples instead of generalising
- Too many training iterations — model keeps fitting noise

How to fix it:

- Get more training data (most effective fix)
- Simplify the model — fewer features, shallower tree, fewer layers
- Regularisation — L1 (Lasso) or L2 (Ridge) — penalise large weights
- Dropout — for neural networks, randomly disable neurons during training
- Cross-validation — detect overfitting early during development
- Early stopping — stop training before the model over-memorises

Think of it this way: A student who memorises every question and answer from past exam papers word-for-word. They score 100% on old papers but fail completely when the exam has new questions — because they memorised, not understood.

DEFINITION**Underfitting**

The model is too simple to capture the actual patterns in the data. It performs poorly on BOTH training data AND test data.

How to spot it:

- Training accuracy: 62% Test accuracy: 60% ← both bad, no big gap = underfitting

Why it happens:

- Model is too simple (fitting a straight line to curved data)
- Too few features — important information is missing from the input
- Training for too few iterations
- Too much regularisation — penalising the model too hard

How to fix it:

- Use a more complex model
- Add more relevant features
- Train for more epochs
- Reduce regularisation strength

Think of it this way: A student who barely studies and tries to guess all answers. They do badly on old papers AND on the actual exam — they never learned enough to begin with.

	Overfitting	Underfitting
Also called	High Variance	High Bias
Train accuracy	Very high (98%)	Low (60%)
Test accuracy	Very low (62%)	Also low (58%)
The gap	Large gap between train & test	No big gap — both bad
Model complexity	Too complex	Too simple
Cause	Too few data, too complex model	Too simple model, too few features
Fixes	More data, regularisation, simpler model	Complex model, more features, less regularisation

■ *A model with 99% training accuracy is NOT necessarily good. Always check test accuracy. A big gap between train and test = overfitting red flag.*

■ Remember

- Overfitting = memorised training data, fails on new data (high variance)
- Underfitting = too simple, fails everywhere (high bias)
- Goal = good performance on BOTH train and test sets

4

Data Preprocessing

Real-world data is messy — clean it before training any model

Why Preprocessing Matters

Raw data from the real world is almost always:

- Incomplete — some values are missing (NaN / NULL)
- Noisy — errors, typos, duplicate rows
- Inconsistent — different units, different formats
- Unscaled — age ranges 0–100, salary ranges 10,000–10,00,000

ML algorithms are mathematical — garbage in, garbage out. If you train on messy data, the model learns messy patterns.

Step 1 — Handling Missing Values

Missing values happen when a row has no value recorded for a column.

Your options:

- Remove the row — only safe when very few rows are missing (< 5%) and missing randomly
- Mean imputation — replace missing number with the column's average
 - Quick and easy, but distorts distribution if many values are missing
- Median imputation — replace with the middle value
 - Better than mean when outliers are present (e.g., salary data with a few billionaires)
- Mode imputation — replace with the most common value
 - Used for categorical columns (e.g., missing 'city' → fill with most common city)
- Model-based imputation — use another ML model to predict the missing value
 - Most accurate but complex to implement

■ **Never fill missing values BEFORE splitting into train/test sets. If you do, test set statistics leak into training — this is data leakage (covered in Section 14).**

Step 2 — Feature Scaling

Many algorithms measure DISTANCE between data points (KNN, SVM, Neural Networks, PCA). If one feature has values 0–1 and another has values 10,000–1,00,000, the large-scale feature completely dominates — the model ignores the small one.

Two main scaling methods:

Method	Formula	Output Range	When to Use
Normalization (Min-Max Scaling)	$(x - \min) / (\max - \min)$	0 to 1 (bounded)	Known min/max bounds, no extreme outliers
Standardization (Z-score Scaling)	$(x - \text{mean}) / \text{std_dev}$	No fixed range, mean=0 std=1	Outliers present; Logistic Reg, PCA, SVM

■ **Tree-based algorithms (Decision Tree, Random Forest, XGBoost) do NOT need feature scaling — they split on thresholds, not distances. Only scale for distance-based or gradient-based algorithms.**

Step 3 — Encoding Categorical Data

ML models only understand numbers. Categories like Male/Female or City names must be converted.

- Label Encoding: Male → 0, Female → 1
 - Only use for Ordinal categories (order matters: Small < Medium < Large)
 - Do NOT use for nominal categories — the model will wrongly think $2 > 1 > 0$
- One-Hot Encoding: creates a new binary column for each category
 - Mumbai → [1,0,0], Delhi → [0,1,0], Chennai → [0,0,1]
 - Safe for nominal categories — no false ordering implied
 - Warning: can explode column count if category has 1000+ values

Step 4 — Feature Engineering & Selection

Feature Engineering: Creating new, more meaningful columns from existing ones.

- Extract year, month, day from a date column
- Create 'BMI' from separate height and weight columns
- Create 'transaction_hour' from timestamp to capture time-of-day patterns

Feature Selection: Removing columns that add noise and slow training.

- Remove columns with near-zero variance (same value in 99% of rows)
- Remove duplicate or highly correlated columns
- Use domain knowledge — does this column actually help predict the target?

■ Remember

- Always split data FIRST, then preprocess each split separately
- Scale for: KNN, SVM, Neural Networks, PCA, Logistic Regression
- Do NOT scale for: Decision Tree, Random Forest, XGBoost
- Label Encoding for ordinal, One-Hot Encoding for nominal categories

5

Supervised Learning Algorithms

Linear Reg · Logistic Reg · Decision Tree · Random Forest · KNN · SVM · Naive Bayes

5.1 Linear Regression

DEFINITION

Linear Regression

Predicts a CONTINUOUS numeric value by fitting the best straight line through the data. Used for regression tasks — price, temperature, salary, patient risk score.

How it works:

- Finds the line $y = w_0x_0 + w_1x_1 + \dots + b$ that minimises prediction errors
- w (weights) = how much each feature influences the output — learned during training
- b (bias) = baseline prediction when all features are zero
- Minimises MSE (Mean Squared Error) — the average of squared differences between predicted and actual

Example: Predict house price from size and location score

- Price = $0.05 \times \text{size} + 500 \times \text{location_score} + 20000$
- These weights are learned automatically from training data

When to use: target is a continuous number, relationship is roughly linear, need interpretability

$$y = w_0x_0 + w_1x_1 + \dots + w_nx_n + b \mid \text{Cost} = (1/n) \times \sum(\text{predicted} - \text{actual})^2$$

Pros	Cons
Simple, fast to train	Assumes linear relationship
Very interpretable — weights show feature importance	Sensitive to outliers
Good baseline — always try it first	Fails on complex non-linear patterns

5.2 Logistic Regression

DEFINITION

Logistic Regression

A CLASSIFICATION algorithm (despite the misleading name) that predicts the probability of a binary outcome (Yes/No, Spam/Not Spam) using the sigmoid function.

Common confusion: The name says 'regression' but it does CLASSIFICATION.

How it works:

- Step 1: Compute $z = w_0x_0 + w_1x_1 + \dots + b$ (same start as linear regression)
- Step 2: Pass through sigmoid function: $P = 1 / (1 + e^{-z})$
 - Sigmoid squashes any number into the range 0 to 1 (a valid probability)
- Step 3: Apply threshold: if $P > 0.5 \rightarrow$ Class 1 (Positive), else Class 0 (Negative)

Threshold tuning — this is powerful:

- Default threshold = 0.5
- Lower threshold (e.g. 0.3) $\rightarrow$ model says YES more often $\rightarrow$ Higher Recall, Lower Precision
 - Use in healthcare: catch more sick patients, even at cost of some false alarms
- Higher threshold (e.g. 0.7) $\rightarrow$ model says YES less often $\rightarrow$ Higher Precision, Lower Recall

- Use in spam filters: only flag obvious spam, don't risk losing real emails

Think of it this way: Think of it like a doctor giving a probability: 'There's a 78% chance this is cancer.' The doctor then applies a threshold: if $> 50\%$ → investigate further. Logistic Regression does exactly this — gives a probability then applies a cutoff.

5.3 Decision Tree

DEFINITION

Decision Tree

Makes predictions by asking a series of yes/no questions about the features, splitting data step by step into increasingly pure groups. Works for both classification and regression.

How it works — a flowchart of questions:

- Root node: first question (e.g., 'Is temperature $> 38^{\circ}\text{C}$?')
- Branch: outcome of each question (Yes / No)
- Internal node: next question on that branch
- Leaf node: final prediction (e.g., 'High Risk' or '0.82 risk score')

How does it choose which question to ask?

It finds the split that creates the purest (most homogeneous) groups:

- Gini Impurity: measures how often a random sample would be wrongly classified
 - Gini = 0 means perfectly pure (all one class)
 - Gini = 0.5 means completely mixed (50/50 split)
- Information Gain (Entropy): measures reduction in disorder after a split
 - Pick the split that reduces entropy the most

Key problem: Decision Trees overfit very easily without limits.

- Without constraints, tree keeps splitting until each leaf has 1 sample → 100% train accuracy, terrible test
- Solution: set max_depth, min_samples_leaf to limit complexity

Think of it this way: Like a doctor playing 20 questions: 'Do you have fever? Yes → Do you have cough? Yes → Likely flu.' Each question narrows down the diagnosis until a conclusion is reached.

5.4 Random Forest

DEFINITION

Random Forest

An ensemble method that builds MANY Decision Trees independently, each trained on a random subset of data and features. Final prediction = majority vote (classification) or average (regression).

The key insight — why many trees beats one tree:

- Each tree sees a different random sample of data (bootstrapping)
- Each tree considers only a random subset of features per split
- Result: each tree makes different mistakes
- When you average many trees' predictions, individual errors cancel out
- The ensemble is far more accurate and robust than any single tree

This technique is called **BAGGING** (Bootstrap AGGregatING):

- Trees built in **PARALLEL** — they do not depend on each other
- Main goal: reduce variance (fix overfitting)

Think of it this way: Instead of asking one doctor for a diagnosis, you ask 100 different doctors (each with slightly different training and experience). The majority diagnosis is far more reliable.

	Decision Tree	Random Forest
# of trees	1	100–1000
Overfitting	Extreme — memorises training	Much reduced
Accuracy	Moderate	High
Interpretability	High — you can draw it	Low — 500 trees hard to visualise
Speed	Very fast	Slower (but parallelisable)

5.5 KNN — K-Nearest Neighbors

DEFINITION

KNN

A lazy learning algorithm that classifies a new data point by finding its K nearest neighbours in the training data and taking a majority vote of their classes.

Why 'lazy'? Because it does **NO** training. It stores all training data and does all work at prediction time.

How prediction works:

- Step 1: Calculate distance from new point to every training point
- Step 2: Find the K closest training points (nearest neighbours)
- Step 3: Majority vote → assign class of the majority

Choosing K:

- $K=1$ → very sensitive to noise, overfits — one wrong neighbour decides everything
- Large K → smoother boundary, might miss local patterns (underfits)
- Rule of thumb: $K = \sqrt{\text{number of training samples}}$, always use odd K to avoid ties
- Tune K using cross-validation

■ **MANDATORY:** Scale features before using KNN. If salary (0–10,00,000) and age (0–100) are both features, salary dominates the distance calculation and age becomes irrelevant. Normalise or standardise all features first.

5.6 SVM — Support Vector Machine

DEFINITION

SVM

Finds the optimal hyperplane that separates two classes with the **MAXIMUM MARGIN** — the widest possible gap between the two class boundaries.

Key concepts:

- Hyperplane: the decision boundary (a line in 2D, a plane in 3D, a hyperplane in higher dimensions)
- Margin: the distance from the hyperplane to the nearest data points on each side

- Support Vectors: the data points closest to the hyperplane that define its position
- Maximising margin = more robust boundary, better generalisation to new data

What if data is NOT linearly separable?

- Use the Kernel Trick — mathematically transform data into higher dimensions where it IS separable
- Linear Kernel: data is linearly separable as-is
- RBF (Radial Basis Function): most common, handles complex circular/curved boundaries
- Polynomial Kernel: useful for image classification tasks

Think of it this way: Imagine separating red and blue marbles on a table. SVM finds the line that puts the most space between the two groups. If marbles are mixed together, SVM imaginarily lifts the table into 3D space where they CAN be separated by a flat plane.

5.7 Naive Bayes

DEFINITION

Naive Bayes

A probabilistic classifier based on Bayes' Theorem that assumes all features are independent of each other. Extremely fast and surprisingly effective for text classification.

Bayes' Theorem in plain English:

Given that this email contains the words 'free money win prize', what is the probability it is spam?

$$P(\text{Spam} \mid \text{these words}) = P(\text{these words} \mid \text{Spam}) \times P(\text{Spam}) / P(\text{these words})$$

- $P(\text{Spam})$ = prior — e.g., 30% of all emails are spam
- $P(\text{words} \mid \text{Spam})$ = how likely are these words IF the email is spam
- $P(\text{Spam} \mid \text{words})$ = posterior — updated belief after seeing the words

The 'Naive' assumption: each word's presence is independent of other words

- In reality this isn't true ('New York' — these words are related)
- Despite this wrong assumption, Naive Bayes works remarkably well for text

Types:

- Multinomial NB: word counts — best for text classification
- Bernoulli NB: binary features — word present or absent
- Gaussian NB: continuous features — assumes normal distribution

$$P(\text{Class} \mid \text{Features}) = [P(\text{Features} \mid \text{Class}) \times P(\text{Class})] / P(\text{Features})$$

Remember

- Linear Reg → continuous number. Logistic Reg → binary class (despite the name!)
- Decision Tree → overfits easily. Random Forest → many trees in parallel fix it
- KNN → must scale features, no training phase, lazy learner
- SVM → max margin hyperplane, kernel trick for non-linear data
- Naive Bayes → Bayes' theorem + independence assumption, best for text

6

Unsupervised Learning Algorithms

K-Means · PCA · Hierarchical Clustering · Anomaly Detection

6.1 K-Means Clustering

DEFINITION

K-Means

Partitions unlabelled data into K distinct clusters by iteratively assigning points to the nearest cluster centre (centroid) and updating those centres.

Step-by-step algorithm:

- Step 1: Choose K — how many clusters you want
- Step 2: Randomly place K centroids in the feature space
- Step 3: Assign each data point to the nearest centroid (Euclidean distance)
- Step 4: Recalculate each centroid = the mean of all points assigned to it
- Step 5: Repeat Steps 3 & 4 until centroids stop moving (convergence)

How to choose K — the Elbow Method:

- Run K-Means with K = 1, 2, 3, 4, 5... and record the WCSS for each
- WCSS (Within-Cluster Sum of Squares) = total distance of all points from their centroid
- Plot WCSS vs K — curve drops steeply then flattens
- The 'elbow' — where the drop slows dramatically — is the optimal K

Limitations:

- Must specify K before running — you need prior knowledge of cluster count
- Sensitive to outliers — one outlier can pull a centroid far from the real cluster
- Assumes clusters are roughly spherical and equal in size
- Different random initialisations can give different results — run multiple times

Think of it this way: Imagine dropping K magnets on a floor scattered with iron filings. Each filing snaps to the nearest magnet. Then each magnet moves to the centre of its group. Repeat until the magnets stop moving.

6.2 PCA — Principal Component Analysis

DEFINITION

PCA

A dimensionality reduction technique that compresses many features into fewer new features (principal components) while retaining as much information as possible.

The problem PCA solves:

- You have 100 features but many are correlated (redundant with each other)
- Training is slow, models overfit, and visualisation is impossible
- PCA compresses 100 features into 5-10 components that capture 95% of the information

How PCA works conceptually:

- Step 1: Standardise all features — PCA is scale-sensitive
- Step 2: Find the directions of maximum variance in the data
 - PC1 (First Principal Component): the direction where data varies the most

- PC2: next most variance, must be perpendicular to PC1
- Step 3: Rank components by how much variance they explain
- Step 4: Keep only top N components explaining ~95% of total variance
- Step 5: Project all data onto these new axes

Important facts:

- PCA is UNSUPERVISED — it only looks at X (features), not y (labels)
- New components are combinations of original features — harder to interpret
- You gain: faster training, less overfitting, ability to visualise in 2D/3D

Think of it this way: Imagine a 3D cloud of data points. PCA finds the best 2D plane to project that cloud onto so you lose the least possible information by flattening it. Like taking the best-angle photograph of a 3D sculpture.

■ **PCA does NOT select original features. It creates NEW features that are linear combinations of originals. You lose interpretability — individual feature meaning is gone.**

6.3 Anomaly Detection

DEFINITION

Anomaly Detection

Identifying data points that are significantly different from the expected pattern — outliers, rare events, or suspicious behaviour.

Use cases:

- Financial fraud: transaction looks very different from customer's usual behaviour
- Medical monitoring: patient vitals suddenly spike outside normal range
- Network security: unusual login times or locations
- Manufacturing: defective products with abnormal measurements

Common approaches:

- Statistical: Z-score or IQR — flag values beyond 3 standard deviations or $1.5 \times$ IQR
- Isolation Forest: anomalies are isolated faster (fewer random splits needed)
- Autoencoder (deep learning): trains on normal data, anomalies reconstruct poorly
- DBSCAN: points in very low density regions = anomalies

■ Remember

- K-Means = assign to K centroids, recalculate, repeat. Use Elbow Method to find K
- PCA = compress many features into fewer components, keeps 95% of variance
- PCA requires standardisation first — it is scale-sensitive
- Anomaly Detection = find points very different from the norm

7

Model Evaluation Metrics

How to actually measure if your model is good — the most-asked interview topic

7.1 The Confusion Matrix — Foundation of All Metrics

DEFINITION**Confusion Matrix**

A 2×2 table that shows exactly how a binary classifier performed — breaking predictions into four categories: TP, TN, FP, FN.

Medical example throughout: predicting if a patient has a disease.

The four cells — understand each one deeply:

- TP (True Positive): Model said 'Has Disease' → Patient ACTUALLY has disease ✓ CORRECT
- TN (True Negative): Model said 'No Disease' → Patient ACTUALLY has no disease ✓ CORRECT
- FP (False Positive): Model said 'Has Disease' → Patient actually DOESN'T have it ✗ WRONG
 - Also called Type I Error or false alarm
 - Patient gets unnecessary tests — costly and stressful but survivable
- FN (False Negative): Model said 'No Disease' → Patient ACTUALLY HAS disease ✗ WRONG
 - Also called Type II Error or a miss
 - Patient goes home untreated — potentially life-threatening

■ *False Negatives are usually more dangerous than False Positives in healthcare. Missing a real sick patient is worse than incorrectly flagging a healthy one.*

7.2 Accuracy — and When It Fails

DEFINITION**Accuracy**

The fraction of total predictions that were correct.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

The imbalanced data trap:

- Hospital dataset: 99% patients are healthy, 1% have a rare disease
- A model that ALWAYS predicts 'Healthy' gets 99% accuracy
- But it catches ZERO sick patients — completely useless for its actual purpose
- Accuracy = 99% looks amazing but the model is broken

■ *On imbalanced data, accuracy is MEANINGLESS. Always check Precision, Recall, and F1.*

7.3 Precision — When False Alarms Are Costly

DEFINITION**Precision**

Of all the cases the model predicted as POSITIVE, what fraction were actually positive? Measures the quality of positive predictions.

Precision = $TP / (TP + FP)$ ← of all cases model said YES, how many were really YES?

High Precision = when the model says 'sick', it's usually actually sick.

Low Precision = the model flags lots of healthy people as sick (many false alarms).

Prioritise Precision when False Positives are costly:

- Spam Filter: marking a real email as spam (FP) means losing an important email
- Legal AI: falsely accusing an innocent person
- Bank fraud alerts: too many alerts irritate customers and waste investigation resources

Trade-off: To increase Precision, raise the classification threshold — but Recall drops.

7.4 Recall — When Missing Cases Is Dangerous

DEFINITION

Recall

Of all the cases that were ACTUALLY POSITIVE, what fraction did the model correctly detect? Also called Sensitivity or True Positive Rate.

Recall = $TP / (TP + FN)$ ← of all real YES cases, how many did the model catch?

High Recall = model catches almost every real positive case.

Low Recall = model misses many real positive cases — dangerous!

Prioritise Recall when False Negatives are dangerous:

- Cancer detection: missing real cancer (FN) could be fatal
- COVID testing: a false negative spreads disease to others
- Fire alarm: missing a real fire is catastrophic
- VigilAI MedLink: missing a high-risk patient's triage could cost a life ← your project!

Trade-off: To increase Recall, lower the threshold — but more false alarms (Precision drops).

7.5 F1 Score — The Balance Between Both

DEFINITION

F1 Score

The harmonic mean of Precision and Recall. Best single metric for imbalanced datasets when BOTH false positives and false negatives matter.

F1 = $2 \times (Precision \times Recall) / (Precision + Recall)$ ← high only when BOTH are high

Why harmonic mean and not regular average?

- Regular average: Precision=1.0, Recall=0.01 → Average = 0.505 (looks OK)
- Harmonic mean: Precision=1.0, Recall=0.01 → F1 = 0.02 (correctly shows terrible)
- Harmonic mean punishes extreme imbalance — you MUST have both high to get a high F1

Metric	Formula	Ask Yourself	Prioritise When
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall % correct?	Balanced data, equal error costs

Precision	$TP/(TP+FP)$	When I say YES, am I right?	False alarms are costly (spam, legal)
Recall	$TP/(TP+FN)$	Did I catch ALL real cases?	Missing cases dangerous (healthcare, VigilAI)
F1 Score	$2 \times P \times R / (P + R)$	Balance of both?	Imbalanced data, both errors matter

7.6 AUC-ROC

DEFINITION

AUC-ROC

Area Under the ROC Curve. Measures how well the classifier separates positive from negative across ALL possible thresholds. Single number summarising overall model quality.

ROC Curve plots True Positive Rate vs False Positive Rate at every threshold 0→1.

Interpreting AUC:

- AUC = 1.0 → perfect model
- AUC = 0.9+ → excellent
- AUC = 0.7–0.9 → good, acceptable for most applications
- AUC = 0.5 → random guessing — no better than a coin flip
- AUC < 0.5 → worse than random (model is inverted)

7.7 Regression Metrics — MAE, MSE, RMSE, R²

Metric	Formula	Units	Key Property	Use When
MAE	$\text{avg}(\text{actual} - \text{pred})$	Same as target	Treats all errors equally, robust to outliers	Outliers present
MSE	$\text{avg}((\text{actual} - \text{pred})^2)$	Target ² (squared)	Penalises large errors very heavily	Internal optimisation
RMSE	$\sqrt{\text{MSE}}$	Same as target	Large errors penalised, interpretable unit	Big errors very costly
R²	$1 - \text{MSE}_{\text{model}} / \text{MSE}_{\text{mean}}$	0 to 1	% variance explained by the model	Want % explanation

Remember

- Confusion Matrix = foundation — TP, TN, FP, FN. All classification metrics derive from it
- Accuracy fails on imbalanced data — always check Precision/Recall/F1 too
- Precision = quality of YES predictions (fewer false alarms)
- Recall = coverage of real YES cases (fewer misses) ← critical for VigilAI
- F1 = best single metric for imbalanced datasets
- Regression: RMSE if big errors costly, MAE if outliers present, R² for % explained

8

Bias, Variance & Regularization

Why models fail and the mathematical tools to fix them

DEFINITION**Bias-Variance Tradeoff**

The fundamental tension in ML: reducing bias (underfitting) tends to increase variance (overfitting) and vice versa. The goal is finding the sweet spot with low bias AND low variance.

Total Error = Bias² + Variance + Irreducible Noise

Bias = how wrong the model is on average (systematic, consistent error)

- High bias = model makes consistent wrong assumptions = underfitting
- Example: fitting a straight line to clearly curved data

Variance = how much predictions change when you train on different data

- High variance = model is unstable, changes a lot with small data changes = overfitting
- Example: a deep Decision Tree that memorises every training sample

Irreducible Noise = random error in the data that no model can ever eliminate

	High Bias (Underfitting)	High Variance (Overfitting)
Model complexity	Too simple	Too complex
Train accuracy	Low	Very high
Test accuracy	Low	Low
The gap	No big gap	Large train-test gap
Fix	More complex model, more features	Regularisation, more data, simpler model

Regularisation — Penalising Complexity

DEFINITION**Regularisation**

Adding a penalty term to the loss function that punishes the model for having large weights. Forces the model to be simpler, reducing overfitting.

New Loss = Original Loss + $\lambda \times$ Penalty

- λ (lambda) = regularisation strength. Larger λ = more penalty = simpler model
- Too large → underfitting. Too small → overfitting. Tune with cross-validation.

L1 Regularisation (Lasso):

- Penalty = sum of ABSOLUTE values of all weights
- Effect: drives many weights exactly to ZERO → automatic feature selection
- Result: a SPARSE model using only the most important features
- Use when: you believe many features are irrelevant

L2 Regularisation (Ridge):

- Penalty = sum of SQUARED values of all weights
- Effect: shrinks ALL weights toward zero, but rarely exactly to zero
- Result: model uses ALL features but with smaller, controlled weights
- Use when: all features are somewhat relevant

Elastic Net: Combines L1 + L2 — gets both feature selection and smooth shrinkage.

Dropout (Neural Networks only):

- During each training step, randomly deactivate a fraction of neurons (typically 20–50%)
- Network cannot rely on any single neuron — learns redundant representations
- Dropout is ONLY active during training — all neurons active at prediction time

Think of it this way: Like studying for an exam with different study partners each session — sometimes your best partner is absent. You learn to understand the material yourself rather than relying on anyone specific.

■ Remember

- Bias = systematic error (underfitting). Variance = sensitivity to training data (overfitting)
- L1 (Lasso) = zeros out irrelevant features (sparse model)
- L2 (Ridge) = shrinks all features equally (dense model)
- Dropout = randomly disable neurons during training to prevent memorisation

9

Gradient Descent & Optimization

How models actually learn — the engine behind every ML algorithm

DEFINITION**Gradient Descent**

An optimization algorithm that iteratively adjusts model weights in the direction that reduces the loss function — like rolling a ball downhill to find the lowest valley.

The big picture:

- A model starts with random weights (bad predictions)
- We measure how wrong it is using a loss function
- Gradient Descent finds which direction to move each weight to reduce the loss
- We make a small step in that direction
- Repeat thousands of times until the loss is minimised (model is well-trained)

The gradient — what is it?

- The gradient = the slope of the loss function at the current weight values
- Positive gradient → loss increases as weight increases → decrease the weight
- Negative gradient → loss decreases as weight increases → increase the weight
- We always move OPPOSITE to the gradient (downhill)

Weight Update: $w = w - \text{learning_rate} \times \text{gradient_of_loss_with_respect_to_w}$

Learning Rate — the most critical hyperparameter:

- Too high: updates overshoot the minimum, loss bounces or diverges — model never converges
- Too low: updates are tiny, training takes forever
- Good starting point: 0.001. Use a scheduler to reduce it over time.

Loss Functions — what are we minimising?

- Regression → MSE: measures average squared prediction error
- Binary Classification → Binary Cross-Entropy: heavily penalises confident wrong predictions
- Multi-class → Categorical Cross-Entropy: compares predicted probability distribution to true label

Type	Data Used Per Update	Speed	Stability	Best For
Batch GD	All training data	Very slow	Very stable	Small datasets
SGD	1 random sample	Very fast	Noisy, unstable	When speed matters, can escape local minima
Mini-batch GD	Small batch (32–256 samples)	Fast	Good balance	Standard in all deep learning frameworks

Remember

- Gradient Descent = take small steps downhill on the loss surface
- Learning rate too high → diverges. Too low → too slow. ~0.001 is a safe start
- Mini-batch GD is the standard used in virtually all deep learning
- Loss function: MSE for regression, Cross-Entropy for classification

10

Neural Networks & Deep Learning

ANNs · Activation Functions · CNN · RNN · LSTM

DEFINITION

Artificial Neural Network (ANN)

A computational model loosely inspired by the brain — layers of interconnected nodes (neurons) that learn to map inputs to outputs through training.

Structure — three types of layers:

- Input Layer: one neuron per feature. Receives the raw data. No computation here.
- Hidden Layer(s): the 'thinking' layers. Each neuron receives weighted inputs from previous layer, applies an activation function, passes result to next layer.
- Output Layer: produces final prediction. One neuron for regression or binary classification. One neuron per class for multi-class.

How one neuron works:

- Step 1: Weighted sum = $w_1 \times \text{input}_1 + w_2 \times \text{input}_2 + \dots + \text{bias}$
- Step 2: Apply activation function (adds non-linearity — allows learning complex patterns)
- Step 3: Pass result to next layer

Training process — Backpropagation:

- Forward pass: input → through all layers → prediction
- Calculate loss: how wrong is the prediction?
- Backward pass: propagate the error back through layers, compute gradients
- Update all weights using gradient descent
- Repeat for many iterations (epochs)

Activation Functions — Why They Matter

Without activation functions, a neural network with 10 layers is mathematically equivalent to a single linear layer — no matter how deep. Activation functions add non-linearity, allowing the network to learn complex patterns.

Function	Formula	Output Range	Use In	Key Property
Sigmoid	$1/(1+e^{-x})$	0 to 1	Output: binary classif.	Outputs probability, causes vanishing gradient in deep networks
Softmax	$e^{x_i} / \sum e^{x_j}$	0–1, sums to 1	Output: multi-class	Each output is probability, all sum to 1
ReLU	$\max(0, x)$	0 to $+\infty$	Hidden layers (default)	Fast, simple. Dead neuron issue for very negative inputs
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	-1 to +1	Hidden layers	Better than sigmoid for hidden layers, still has vanishing gradient

■ Use ReLU for hidden layers (fast, simple). Sigmoid for binary output layer. Softmax for multi-class output layer. Tanh when negative values matter in hidden layers.

CNN — Convolutional Neural Networks

DEFINITION**CNN**

A deep learning architecture designed for spatial data like images. Uses filters (kernels) that slide over the image to automatically detect features.

Key layers:

- Convolutional Layer: slides small filters over the image to detect local features
 - Early layers detect: edges, corners, lines
 - Deeper layers detect: shapes, textures, objects, faces
- Pooling Layer: reduces spatial dimensions ($28 \times 28 \rightarrow 14 \times 14$)
 - Max Pooling: keeps the maximum value in each region
 - Reduces computation and makes the model location-invariant
- Fully Connected Layer: at the end — flattened features $\rightarrow$ final classification

Use cases: image classification, object detection, face recognition, medical imaging

Think of it this way: Like a human looking at a photo. First your eyes detect edges and basic shapes. Then your brain combines those into objects (nose, ear, eye). Finally it recognises the whole face. CNN layers do this hierarchically, from simple to complex.

RNN — Recurrent Neural Networks

DEFINITION**RNN**

A neural network designed for sequential data where the current output depends on previous inputs. Maintains a hidden state that carries information across time steps.

Key idea: at each time step, the RNN receives the current input AND the hidden state from the previous step. This 'memory' lets it understand context in sequences.

Example: predicting the next word in 'The cat sat on the ____'

- The word 'mat' depends on understanding all previous words
- RNN passes information from word to word across the sequence

The vanishing gradient problem:

- During backpropagation through long sequences, gradients are multiplied together repeatedly
- Very small numbers multiplied many times $\rightarrow$ approaches zero (vanishes)
- Result: the model forgets information from the distant past
- An RNN cannot reliably remember what happened 50+ steps ago

LSTM — Long Short-Term Memory

DEFINITION**LSTM**

An advanced RNN that solves the vanishing gradient problem using learnable gates that control what information to keep, forget, or output at each step.

LSTM has a separate Cell State — a long-term memory highway running through the sequence.

The three gates (all learned from data):

- Forget Gate: decides what old information to erase from cell state
 - 'I've been tracking Topic A for 10 steps, I can forget Topic B now'
- Input Gate: decides what new information to add to cell state

- 'This new information about Topic C is important, store it'
- Output Gate: decides what to output at this time step
- 'Based on current context, output this hidden state to the next layer'

Result: LSTMs can reliably remember relevant information across 100s of time steps

Use cases: time-series forecasting, text generation, machine translation, speech recognition

Your VigilAI project: used LSTM for predicting patient deterioration from sequential vitals data

*Think of it this way: RNN = a person with very short-term memory, forgets what happened 5 minutes ago.
LSTM = a person with a notepad — they decide what to write down, what to erase, and what to read out.
They can remember important context from much earlier.*

■ Remember

- ANN = layers of neurons with activation functions, trained via backpropagation + gradient descent
- ReLU for hidden layers. Sigmoid for binary output. Softmax for multi-class output
- CNN = best for images — filters detect features hierarchically from simple to complex
- RNN = sequential data but forgets long context (vanishing gradient problem)
- LSTM = fixes RNN's memory problem with forget/input/output gates

11

XGBoost — Deep Dive

The algorithm you built VigilAI with — understand it inside-out

DEFINITION**XGBoost
(eXtreme Gradient
Boosting)**

A powerful ensemble algorithm that builds Decision Trees one after another SEQUENTIALLY, where each new tree specifically corrects the mistakes of all previous trees. One of the most accurate algorithms for structured/tabular data.

Understanding Boosting — the core idea:

- Tree 1: trained on original data → makes some predictions, has some errors
- Tree 2: trained specifically on the RESIDUAL ERRORS of Tree 1 → corrects those mistakes
- Tree 3: trained on remaining errors after Trees 1+2 → corrects those
- ... repeat for N trees (typically 100–1000)
- Final prediction = weighted combination of ALL trees' predictions

Why 'Extreme Gradient'?

- 'Gradient': uses gradient descent to find which direction to build the next tree
- Each new tree is built in the direction that most reduces the loss
- 'Extreme': highly optimised with regularisation, parallel processing, efficient data structures

Think of it this way: A team of students all solving the same problem. Student 1 attempts it and makes mistakes. Student 2 specifically focuses on Student 1's mistakes and fixes them. Student 3 fixes what Student 2 got wrong. Each specialist builds on the previous. The combined team is far better than any individual.

Boosting vs Bagging — The Critical Distinction

	Bagging (Random Forest)	Boosting (XGBoost)
How trees are built	In PARALLEL — fully independent	SEQUENTIALLY — each depends on previous
What each tree learns	Random subset of data	Residual errors of previous trees
Main goal	Reduce Variance (fix overfitting)	Reduce Bias first, then variance
Trees are weighted	Equal weight in final vote	Weighted by their accuracy
Speed	Faster (parallel)	Slower (sequential, must wait)
Typical accuracy	Good	Usually better
Risk	Slightly underfits on complex data	Can overfit if not regularised

Why XGBoost Over a Single Decision Tree?

	Decision Tree	XGBoost
# of trees	1	100–1000
Overfitting	Extreme	Controlled via built-in L1/L2 regularisation

Missing values	Must handle manually	Handles natively — learns optimal direction
Accuracy	Moderate	High to very high
Feature importance	Not reliable	Built-in, very useful for explainability
Speed	Very fast	Fast (parallelised within each tree build)

Key XGBoost Hyperparameters — What They Mean

`n_estimators`: number of trees to build

- Typical: 100–1000. More trees = better up to a point, then overfitting risk

`learning_rate` (`eta`): how much each tree contributes to the final model

- Small learning rate (0.01–0.1) + many trees = best generalisation
- Large learning rate + few trees = faster training but may overfit

`max_depth`: maximum depth of each individual tree

- Typical: 3–8. Deeper trees = more complex patterns but more overfit risk

`subsample`: fraction of training data to use per tree (adds randomness like bagging)

- Values 0.6–0.9. Reduces variance, prevents overfitting

`reg_alpha` (L1) and `reg_lambda` (L2): built-in regularisation

- Increase to reduce overfitting. Decrease if model underfits.

Why XGBoost Was Right for VigilAI

In VigilAI MedLink, XGBoost was the best choice because:

- Data was structured and tabular: patient vitals, symptom flags, age, clinical measurements
- Dataset size: small-to-medium — deep learning needs millions of samples to beat XGBoost here
- Interpretability: feature importance shows doctors WHICH symptoms drive the risk score most
- Missing values: patient records often have gaps — XGBoost handles them automatically
- Speed: predictions needed to be fast for real-time triage — XGBoost is very efficient
- Proven: XGBoost consistently wins healthcare ML competitions on Kaggle

Output: Risk score 0.0 to 1.0 (XGBoost in regression mode)

Threshold applied: risk score > 0.6 → High Priority triage flag sent via WebSocket to hospital

■ Remember

- Boosting = sequential trees, each corrects previous errors — different from bagging (parallel)
- XGBoost has: built-in regularisation, handles missing values, feature importance, fast training
- Tune `n_estimators` together with `learning_rate` — low LR + many trees = best generalisation
- For VigilAI: tabular data + small dataset + interpretability needed = XGBoost is ideal choice

12

NLP — Natural Language Processing

Tokenization · TF-IDF · Sentiment Analysis · Text Classification

DEFINITION**NLP (Natural Language Processing)**

The field of AI that enables computers to read, understand, and generate human language. Computers understand numbers — NLP converts text into numbers that models can process.

The core challenge: Language is ambiguous, contextual, and constantly evolving.

- Same word, different meanings: 'bank' — river bank or financial bank?
- Sarcasm: 'Oh great, the server crashed again' — positive words, negative meaning
- Context: 'I saw her duck' — did she duck down or is she holding a duck?

Real-world NLP applications you use every day:

- Gmail: spam filter, smart reply suggestions
- Google Translate: machine translation across 100+ languages
- Siri/Alexa: voice recognition and intent understanding
- ChatGPT/Claude: text generation and question answering
- Amazon reviews: automatic sentiment analysis
- LinkedIn: job recommendation from your resume text

The NLP Pipeline — Every Project Follows This

Raw Text → Tokenize → Remove Stopwords → Stem/Lemmatize → Vectorize → ML Model → Output

Step 1: Tokenization**DEFINITION****Tokenization**

Breaking raw text into smaller units called tokens. The **FIRST** step in every NLP pipeline — no ML model can process raw text directly.

Example: 'I love machine learning' → ['I', 'love', 'machine', 'learning']

Types of tokenization:

- Word tokenization: split on spaces/punctuation
 - Simple but struggles with unknown words (new slang, technical terms)
- Character tokenization: split into individual characters ['I', ' ', 'l', 'o', 'v', 'e'...]
- Handles any text but loses word-level meaning, very long sequences
- Subword tokenization: split into word pieces ['I', 'love', 'ma', 'chine', 'learn', 'ing']
 - Used by ALL modern LLMs (GPT, BERT, Claude)
 - Handles unknown words by splitting them into known subwords
 - Best balance: meaningful units + can process any input

Related preprocessing done after tokenization:

- Stopword removal: delete very common words (is, the, a, an, of, in) — they add no meaning
- Lowercasing: 'Apple' and 'apple' should be the same token
- Punctuation removal: usually stripped

Step 2: Stemming vs Lemmatization

Both reduce words to their root form so variations are treated as the same word.

'running', 'runs', 'ran' → all mean the concept 'run'

Stemming (crude, fast):

- Chops off word endings using simple rules — fast but sometimes creates non-words
- 'running' → 'run' ✓, 'studies' → 'studi' ✗ (wrong), 'better' → 'bett' ✗ (wrong)

Lemmatization (accurate, slower):

- Uses a dictionary to find the correct base form
- 'running' → 'run' ✓, 'better' → 'good' ✓, 'studies' → 'study' ✓
- Preferred when accuracy matters more than raw speed

Step 3: Vectorization — Text to Numbers

ML models need numbers. After preprocessing, text must be converted to numeric vectors.

Bag of Words (BoW) — simplest:

- Build a vocabulary of all unique words across all documents
- Each document = vector of word counts
- 'I love ML' → [0, 0, 1, 1, 1, 0, 0, ...] (1 for each word present)
- Completely ignores word ORDER — just a bag of counts, no context

TF-IDF (Term Frequency — Inverse Document Frequency) — smarter:

- Gives HIGH score to words that are FREQUENT in one document but RARE overall
- 'the' appears in every document → low IDF → low TF-IDF (filtered as unimportant)
- 'triage' appears in very few medical documents → high IDF → high TF-IDF (important)
- Better at identifying truly meaningful, distinctive words in each document

Word Embeddings (Word2Vec, GloVe) — deep approach:

- Each word is a dense vector of 100–300 numbers learned from a massive corpus
- Similar words are CLOSE in vector space — captures semantic meaning
- Famous example: king – man + woman ≈ queen (vector arithmetic actually works!)
- Captures meaning, not just frequency

$$\text{TF-IDF}(\text{word}, \text{doc}) = \text{TF}(\text{word in this doc}) \times \log(\text{Total docs} / \text{Docs containing this word})$$

Sentiment Analysis

DEFINITION

Sentiment Analysis

Automatically classifying the emotional tone of text as Positive, Negative, or Neutral. The most common and practical NLP task in business applications.

How it works:

- Train a classifier on thousands of labelled reviews (5-star = Positive, 1-star = Negative)
- Model learns which words and patterns associate with each sentiment
- Predict sentiment of new, unlabelled text

Challenges:

- Sarcasm: 'Oh brilliant, the app crashed again' — positive words, negative sentiment
- Negation: 'Not bad at all' → Positive (but 'not' + 'bad' looks negative to simple models)
- Mixed: 'Food was great but service was terrible' — contains both positive and negative
- Domain-specific: 'This stock is sick!' might be positive slang in finance

VigilAI context: NLP can analyse free-text symptom descriptions entered by clinics to extract structured features for the XGBoost risk prediction model.

Text Classification

DEFINITION

Text Classification

Assigning a predefined category label to a piece of text.
Sentiment Analysis is a special case of text classification.

Examples:

- Email → Spam / Not Spam
- News article → Sports / Politics / Technology / Entertainment
- Customer complaint → Billing / Technical / Delivery / Other
- Medical note → Urgent / Routine / Follow-up

Algorithms used:

- Naive Bayes: fast probabilistic baseline, great for text, works with small data
- Logistic Regression: simple, interpretable, strong baseline for text
- SVM: works very well for high-dimensional text feature spaces
- Deep Learning (LSTM, BERT): when large labelled data available and accuracy is critical

■ Remember

- NLP Pipeline: Tokenize → Remove stopwords → Lemmatize → Vectorize → Model
- Tokenization = first step always — converts raw text into processable units
- Lemmatization > Stemming (accurate root vs fast but error-prone)
- TF-IDF = high score for words rare globally but frequent in this specific document
- Sentiment Analysis = Positive/Negative/Neutral classification of text

13

LLMs, RAG & Prompt Engineering

GPT · Claude · Hallucination · Fine-tuning · Agents — explained simply

DEFINITION**LLM (Large Language Model)**

A massive AI model trained on billions of words of text. It learns to predict the next word so well that it can write, reason, code, and answer questions across almost any topic — without being retrained for each specific task.

How LLMs are different from traditional ML:

- Traditional ML: trained for ONE specific task (spam detection, price prediction)
- LLM: trained on GENERAL text → can do MANY tasks without task-specific retraining

The architecture: LLMs are based on the Transformer architecture (2017 Google paper)

- Key innovation: Attention mechanism — model learns which words to focus on for each prediction
- Example: 'The cat sat on the mat because IT was tired'
- Attention identifies that IT refers to 'the cat' — not 'the mat'
- Scales efficiently to billions of parameters on GPUs

What can LLMs do?

- Write code, essays, emails, reports, stories
- Answer questions and explain complex concepts
- Translate languages
- Summarise long documents
- Extract structured data from unstructured text
- Generate synthetic training data for other ML models

LLM	Company	Key Characteristic
GPT-4 / ChatGPT	OpenAI	Most widely used, very strong at reasoning and coding
Gemini	Google DeepMind	Integrated with Google products, multimodal
Claude	Anthropic	Long context window, safety-focused, nuanced responses
Llama 3	Meta	Open-source — can be downloaded and run locally
Mistral	Mistral AI	Small but powerful open-source model, fast inference

Context Window — What It Means

The context window is the maximum amount of text an LLM can read and consider at once.

- Early models: ~4,000 tokens (~3,000 words)
- Modern models: 128,000+ tokens (~100,000 words — an entire book)
- Everything in your conversation must fit inside the context window
- If a conversation exceeds the context window, the model 'forgets' the earliest messages
- This is why very long conversations with LLMs can lose context from the beginning

Hallucination — The Key Weakness of LLMs

DEFINITION**Hallucination**

When an LLM confidently generates factually incorrect information that sounds completely believable and authoritative. The model doesn't know what it doesn't know.

Why hallucination happens:

- LLMs are trained to generate statistically LIKELY next words — not to verify facts
- They don't have access to a fact database — they 'remember' from training data patterns
- Training data has errors, biases, and outdated information
- The model generates what SOUNDS correct given the context, not what IS correct

Examples of hallucination:

- Inventing fake citations: 'According to Smith et al. 2019...' — the paper doesn't exist
- Wrong historical facts given confidently with specific (wrong) dates
- Outdated information: LLMs have knowledge cutoffs and don't know recent events
- Invented statistics or figures that sound plausible but are made up

How to reduce hallucination:

- Use RAG — give the model real, verified documents to answer from
- Prompt the model explicitly: 'If you don't know, say I don't know'
- Use temperature=0 for factual tasks (less randomness = more consistent output)
- Always verify LLM outputs against trusted sources for critical decisions

■ *Never use LLM outputs directly in medical, legal, or financial contexts without human verification. LLMs can be confidently, fluently, completely wrong.*

Prompt Engineering

DEFINITION**Prompt Engineering**

The skill of writing clear, structured instructions to an LLM to get accurate, useful, and consistent responses. The way you phrase your input directly affects output quality.

Why it matters: the exact same model gives terrible or excellent answers depending entirely on how you phrase the question.

1. Zero-Shot Prompting — ask directly, no examples:

- Weak: 'Explain ML' → likely generic, too brief
- Better: 'Explain machine learning in 3 bullet points for a CSE fresher preparing for interviews'

2. Few-Shot Prompting — give 2-3 examples first, then ask:

- Show 3 examples of the format/style you want → model follows the pattern precisely

3. Chain-of-Thought — ask the model to 'think step by step':

- 'Solve this problem step by step.' → better for math, reasoning, multi-step problems

4. System Prompt — set the model's role and constraints:

- 'You are a medical triage assistant. Never diagnose. Always recommend consulting a doctor.'
- Consistent behaviour maintained across all user messages in the conversation

5. Structured Output — ask for specific format:

- 'Respond ONLY in JSON with keys: risk_level, symptoms, recommended_action'
- Makes LLM output parseable by your code without additional processing

RAG — Retrieval-Augmented Generation

DEFINITION

RAG

A technique where real documents from your own database are retrieved and given to the LLM as context before it answers. The model answers from YOUR documents, not its training memory.

The problem RAG solves:

- LLM trained on general internet data — knows nothing about YOUR specific documents
- LLM knowledge has a cutoff date — doesn't know about recent events
- LLM might hallucinate when it doesn't know the answer from training

How RAG works — step by step:

- Step 1: User asks a question
- Step 2: RAG system converts the question into a numeric vector (embedding)
- Step 3: Searches your document database for the most similar document vectors
- Step 4: Top 3-5 most relevant document chunks are retrieved
- Step 5: These documents + the original question are bundled into the LLM prompt
- Step 6: LLM reads the provided documents and answers ONLY from what's in them
- Result: answer is grounded in your real documents — much less hallucination

VigilAI application: RAG could power a clinical decision support chatbot where doctors ask questions and the system retrieves relevant patient history + medical guidelines to answer from.

	RAG	Fine-Tuning
What changes	Nothing in the model — docs fed at query time	Model weights updated (retrained)
Cost	Low — just a vector database	High — needs GPU, training data
Update content	Easy — add/change documents anytime	Hard — must retrain from scratch
Accuracy	High on your specific documents	Learns style/domain deeply
Hallucination	Greatly reduced	Still possible
When to use	Answer questions from your documents	Need model to learn new style or terminology

Agentic AI

DEFINITION

Agentic AI

AI that doesn't just answer questions — it autonomously takes sequences of actions using tools (web search, code execution, APIs, databases) to complete multi-step tasks.

Traditional LLM: User asks → LLM answers → Done (single turn)

Agentic AI: User gives goal → Agent plans steps → Agent uses tools → Agent verifies results → Delivers output

Tools an AI agent can use:

- Web search: look up current information the model doesn't know

- Code execution: write and run Python code to analyse data or solve problems
- Database queries: retrieve specific records
- API calls: book a calendar event, send an email, check weather

Multi-Agent Systems: multiple specialised agents collaborating

- Orchestrator Agent: breaks down the task, assigns sub-tasks to specialist agents
- Research Agent: searches for relevant information
- Analysis Agent: processes and interprets the retrieved data
- Writing Agent: compiles everything into a final coherent output

■ Remember

- LLM = trained on billions of words, can do many tasks, based on Transformer + Attention
- Hallucination = confident wrong answers — always verify critical LLM outputs
- Prompt Engineering = better instructions → dramatically better LLM answers
- RAG = retrieve real documents → feed to LLM → grounded answers without retraining
- RAG (cheap, easy to update) vs Fine-tuning (expensive, changes model behaviour)

14

Practical ML

Imbalanced data · Data leakage · Hyperparameter tuning

Handling Imbalanced Datasets

DEFINITION

Imbalanced Dataset

A dataset where one class has far more samples than another — e.g., 99% healthy, 1% disease. Standard models default to predicting the majority class always.

The problem:

- Model trained on 99% healthy data learns to almost always predict 'Healthy'
- Gets 99% accuracy — looks amazing on paper
- But it catches ZERO sick patients — completely useless for the actual task

Solutions:

1. Oversampling — increase minority class samples:

- Random Oversampling: duplicate minority samples — quick but adds no new information
- SMOTE (Synthetic Minority Over-sampling Technique) — BEST PRACTICE:
 - Generates synthetic new minority samples by interpolating between existing ones
 - Creates realistic new examples, not just duplicates
 - New sample = random point on the line between two real minority samples

2. Undersampling — decrease majority class:

- Remove some majority class samples until classes are balanced
- Risk: losing important majority class information
- Only use when majority class is very large and you can afford to remove data

3. Class Weights — penalise minority class errors more heavily:

- Set `class_weight='balanced'` in sklearn — minority errors cost more
- Simple, no data modification needed, works in most algorithms

4. Better Metrics — stop using Accuracy:

- Switch to F1 Score, Recall, AUC-ROC
- These expose the model's failure to predict the minority class

■ *Apply SMOTE ONLY on training data, NEVER on test data. Synthesising test data means you evaluate on fake examples — not a real measure of production performance.*

Data Leakage

DEFINITION

Data Leakage

When information from outside the training set accidentally influences the model during training, causing artificially inflated performance that completely fails in production.

The classic mistake — normalisation before splitting:

- You compute mean and std from the FULL dataset (train + test combined)
- Then you split into train and test
- Problem: the test set's values influenced the normalisation statistics

- Your model has 'seen' the test set indirectly — performance is dishonestly inflated

The correct order — ALWAYS:

- Step 1: Split data into train and test FIRST, before any preprocessing
- Step 2: Fit your scaler on TRAINING data only (compute mean and std from train)
- Step 3: Use that same scaler to transform BOTH train and test separately
- The test set must be completely isolated until the very final evaluation

Other common leakage causes:

- Feature engineering using the target: 'average target value for this customer' leaks label info
- Temporal leakage: predicting Monday's diagnosis using Tuesday's blood test result
- Duplicate rows: same patient in both train and test → model has 'seen' the test case

WRONG: `normalize(all data)` → `split` | **CORRECT:** `split` → `normalize(train only)` → `transform(train and test)`

Hyperparameter Tuning

Two types of model parameters — understand the difference:

- Parameters: values the MODEL learns from data (weights, biases) — you don't set these
- Hyperparameters: values YOU set before training that control the learning process

Examples of hyperparameters:

- Learning rate, n_estimators, max_depth, K in KNN, C in SVM, dropout rate, batch size

Tuning methods:

Grid Search:

- Define a grid: n_estimators=[100,200,500], max_depth=[3,5,7]
- Try EVERY combination: (100,3), (100,5), (100,7), (200,3)... all 9 combinations
- Best combination = winner
- Problem: grows exponentially as you add more hyperparameters

Random Search:

- Randomly sample N combinations from the hyperparameter space
- Usually finds near-optimal in much less time than grid search
- Preferred when you have many hyperparameters to tune

Bayesian Optimization:

- Learns from previous trials which hyperparameter values are promising
- Focuses exploration on most promising regions — most efficient
- More complex to set up but widely used in production

ALWAYS use cross-validation during tuning to avoid overfitting to the validation set.

■ Remember

- Imbalanced data: use SMOTE on training data only. Switch to F1/Recall/AUC metrics
- Data leakage: split FIRST, then preprocess. Test set must stay completely isolated
- Hyperparameters = settings YOU choose before training. Parameters = model learns from data
- Random Search usually beats Grid Search in speed with similar accuracy



Quick Revision + Top 30 Q&A;

Read this the night before your interview

ML BASICS

- ML = system learns rules from data automatically, no manual rule-writing
- AI $\supset$ ML $\supset$ Deep Learning — nested circles, not synonyms
- Supervised = labelled data. Unsupervised = no labels. Reinforcement = reward/penalty
- Overfitting = memorised train data, fails on test (high variance). Fix: more data, regularisation
- Underfitting = too simple, fails everywhere (high bias). Fix: more complex model

ALGORITHMS

- Linear Regression $\rightarrow$ predicts continuous numbers (price, score)
- Logistic Regression $\rightarrow$ predicts classes (despite the name — it is a CLASSIFIER)
- Decision Tree $\rightarrow$ overfits easily. Random Forest $\rightarrow$ many parallel trees fix overfitting
- KNN — lazy learner, no training phase, MUST scale features (distance-based)
- SVM $\rightarrow$ max margin hyperplane. Kernel trick for non-linear data
- Naive Bayes $\rightarrow$ Bayes theorem + independence assumption, great for text classification

METRICS

- Accuracy fails on imbalanced data — always check Precision/Recall/F1 too
- Precision = $TP/(TP+FP)$: when model says YES, how often correct? (spam, legal)
- Recall = $TP/(TP+FN)$: of all real YES, how many caught? (healthcare, VigilAI)
- F1 = harmonic mean of Precision+Recall. Best for imbalanced datasets
- Confusion Matrix: TP (correct yes), TN (correct no), FP (false alarm), FN (dangerous miss)

XGBOOST

- Sequential trees — each corrects previous errors = Boosting
- Random Forest = Bagging (parallel, reduce variance). XGBoost = Boosting (sequential, reduce bias)
- Built-in: handles missing values, L1/L2 regularisation, feature importance
- Best for structured tabular data with small-to-medium datasets
- VigilAI: tabular patient data + interpretability needed + small dataset = XGBoost ideal

NLP

- Pipeline: Tokenize → Remove stopwords → Lemmatize → Vectorize (TF-IDF) → Model
- Tokenization = FIRST step always. Converts raw text into processable units
- Lemmatization = accurate root ('better'→'good'). Stemming = crude cut ('better'→'bett')
- TF-IDF = high score for words rare globally but frequent in this specific document
- Sentiment Analysis = Positive/Negative/Neutral classification of text

LLMs & RAG

- LLM = massive model on billions of words, can do many tasks, Transformer+Attention architecture
- Hallucination = LLM confidently gives wrong answers. Always verify critical outputs
- Prompt Engineering = better instructions → dramatically better LLM answers
- RAG = retrieve real documents → feed to LLM → grounded answers without retraining
- RAG vs Fine-tuning: RAG for your documents (cheap). Fine-tuning to change model behaviour (expensive)

Top 30 Interview Questions & Answers

The most commonly asked questions for freshers in AI/ML rounds. Read each answer out loud — saying it is the best way to prepare.

Q1 What is Machine Learning?

ML is a way of teaching computers to learn patterns from data automatically, without a human writing the rules. You provide examples (data + correct answers), the computer figures out the rules itself, and uses them to predict outcomes for new inputs.

Q2 What are the three types of ML?

Supervised Learning — trained on labelled data (correct answers provided). Unsupervised Learning — no labels, model finds hidden patterns or groups. Reinforcement Learning — agent learns by taking actions and receiving rewards or penalties from the environment.

Q3 What is the difference between AI, ML, and Deep Learning?

AI = any technique enabling machines to simulate intelligent behaviour (broadest circle). ML = a subset of AI where machines learn from data without explicit programming. DL = a subset of ML using multi-layered neural networks for very complex patterns like images and speech.

Q4 What is overfitting? How do you prevent it?

Overfitting = the model memorises training data including its noise, so it performs great on training but poorly on new data. Prevention: get more data, simplify the model, use regularisation (L1/L2), cross-validation, or dropout in neural networks.

Q5 What is underfitting?

Underfitting = the model is too simple to capture the patterns — performs poorly on BOTH training AND test data. Fix: use a more complex model, add more relevant features, train for more epochs, reduce regularisation strength.

Q6 What is the difference between classification and regression?

Classification predicts a discrete category label (spam/not spam, disease/healthy, cat/dog). Regression predicts a continuous number (price, temperature, patient risk score 0 to 1). Decision Tree and XGBoost can handle both tasks.

Q7 What are features and labels?

Features (X) = the input variables describing each data point — symptoms, age, vitals in VigilAI. Label (y) = the correct output the model is trained to predict — risk score or disease class. ML = learning to map X to y automatically from training examples.

Q8 What is cross-validation?

Technique to get a reliable estimate of model performance. Split data into K folds. Train on K-1 folds, test on 1 fold. Repeat K times — each fold is the test set exactly once. Average the K performance scores. More reliable than a single train/test split.

Q9 What is a confusion matrix?

A 2x2 table showing exactly how a binary classifier performed. TP = correctly predicted positive. TN = correctly predicted negative. FP = predicted positive but actually negative (false alarm). FN = predicted negative but actually positive — the most dangerous error in healthcare.

Q10 What is Accuracy, and when should you NOT use it?

Accuracy = $(TP+TN)/(TP+TN+FP+FN)$ — fraction of correct predictions. Do NOT use it on imbalanced data. A model always predicting the majority class gets high accuracy but catches zero minority cases. Use Precision, Recall, or F1 instead.

Q11 What is Precision?

Precision = $TP/(TP+FP)$. Of all predictions the model made as positive, how many were actually positive? High Precision means low false alarm rate. Prioritise when false positives are costly — spam filters, legal AI, fraud alert systems.

Q12 What is Recall?

Recall = $TP/(TP+FN)$. Of all actual positive cases, how many did the model correctly catch? High Recall means few missed real cases. Prioritise when false negatives are dangerous — cancer detection, COVID testing, disease triage. In VigilAI MedLink.

Q13 What is F1 Score?

F1 = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ — the harmonic mean of both metrics. High F1 only when BOTH Precision AND Recall are high. Best single metric for imbalanced datasets when both false positives and false negatives matter.

Q14 For VigilAI, which metric is most important and why?

Recall is most critical. Missing a high-risk patient (false negative) could be life-threatening — the patient goes home without proper triage. A false alarm (false positive) causes extra investigation but the patient remains safe. In healthcare, missing a real case is always more dangerous.

Q15 What is AUC-ROC?

AUC = Area Under the ROC Curve. ROC plots True Positive Rate vs False Positive Rate at every possible threshold. AUC = 1.0 is perfect, AUC = 0.5 is random guessing. Threshold-independent — gives overall model quality as a single number.

Q16 What is XGBoost?

eXtreme Gradient Boosting. Builds Decision Trees sequentially — each new tree specifically corrects the errors of all previous trees combined. Very high accuracy on structured tabular data, handles missing values natively, has built-in regularisation, and provides feature importance scores.

Q17 What is the difference between Bagging and Boosting?

Bagging (Random Forest): builds trees in PARALLEL, each on a random data subset, takes majority vote. Goal = reduce variance. Boosting (XGBoost): builds trees SEQUENTIALLY, each corrects previous errors, weighted combination. Goal = reduce bias then variance. Boosting usually achieves higher accuracy.

Q18 Why did you choose XGBoost for VigilAI?

Tabular patient data (vitals, symptoms, age) — XGBoost excels here. Small-to-medium dataset — deep learning needs millions of samples to beat XGBoost in this regime. Handles missing patient values natively. Feature importance tells doctors which symptoms drive the risk score most.

Q19 What is feature importance in XGBoost?

XGBoost calculates how much each input feature contributed to reducing prediction error across all trees. Gives a ranking score per feature — you can see which symptoms or vitals are most predictive of high risk. Critical for medical interpretability and building clinical trust in the model.

Q20 What is regularisation in XGBoost?

XGBoost has L1 (reg_alpha) and L2 (reg_lambda) regularisation built in. These penalise large weights in trees, preventing individual trees from overfitting. Also has subsample (use fraction of data per tree) and max_depth (limit tree complexity) as additional regularisation controls.

Q21 What is NLP? Give 3 real applications.

NLP = Natural Language Processing — AI enabling computers to understand and work with human language. Applications: (1) Gmail spam filter, (2) Google Translate, (3) ChatGPT/Claude, (4) Amazon review sentiment analysis, (5) Siri/Alexa voice assistants.

Q22 What is tokenization? Why is it the first step?

Breaking raw text into smaller units called tokens. The first step in every NLP pipeline — ML models cannot process raw text strings directly. They need numbers. Tokenization converts text into pieces that can then be converted to numeric vectors for the model.

Q23 What is the difference between stemming and lemmatization?

Stemming = crude rule-based cut to root. 'better' → 'bett' (wrong), 'studies' → 'studi' (wrong). Fast but error-prone. Lemmatization = dictionary-based accurate root. 'better' → 'good' (correct), 'studies' → 'study' (correct). Slower but accurate. Prefer lemmatization for quality NLP.

Q24 What is TF-IDF?

Term Frequency–Inverse Document Frequency. Gives high score to words that are frequent in ONE specific document but RARE across all documents. Helps identify the most meaningful and distinctive words. 'the' is everywhere so low TF-IDF. 'triage' appears rarely so high TF-IDF in medical documents.

Q25 What is sentiment analysis? What are the challenges?

Classifying text as Positive, Negative, or Neutral based on emotional tone. Used for product reviews, social media, customer feedback. Key challenges: sarcasm ('Oh brilliant, it crashed again'), negation ('Not bad' = positive), mixed sentiment ('Food great, service terrible').

Q26 What is an LLM? Name three examples.

Large Language Model — trained on billions of words of text, can understand and generate human-like text across many tasks without being retrained for each. Examples: GPT-4 by OpenAI, Gemini by Google, Claude by Anthropic, Llama 3 by Meta (open-source), Mistral by Mistral AI.

Q27 What is hallucination in LLMs? How do you reduce it?

Hallucination = LLM generates confident, fluent, but factually wrong information. Happens because LLMs generate statistically likely text, not verified facts. Solutions: use RAG (give real documents to answer from), instruct model to say 'I don't know' when uncertain, always verify critical outputs.

Q28 What is RAG? How does it work?

Retrieval-Augmented Generation. User asks a question → system retrieves the most relevant documents from your database → these documents + the question are given to the LLM as context → LLM answers from the real provided documents, not training memory. Dramatically reduces hallucination.

Q29**What is the difference between RAG and Fine-Tuning?**

RAG: feed documents at query time, model weights unchanged. Cheap, easy to update, answers from your documents. Fine-tuning: update model weights on domain data. Expensive, powerful, changes how the model behaves and what it knows. Use RAG when you want to answer from your docs. Use Fine-tuning when you need to change the model's style or domain knowledge.

Q30**What is Prompt Engineering?**

The skill of writing clear, specific instructions to get better LLM outputs. Techniques: Zero-shot (ask directly), Few-shot (give 2-3 examples first), Chain-of-thought (ask to think step by step), System prompts (set the model's role and constraints), Structured output (ask for specific JSON format). Better prompts dramatically improve response quality without changing the model.
